

proof; hence, security designers need to explore all possible options prior to putting appropriate controls in place, leading to a robust design. This is where understanding and using the in-built security features of FOSS distros becomes important—to introduce robustness.

## FOSS security features

While there are so many distros available with various built-in features, I will concentrate on those features that are found in almost all versions. Some of the features mentioned below are actually open source projects that became integral parts of distros, over time.

**Iptables:** All Linux distros support iptables, which is essentially a truth-table sort of database containing information that lets the net-filter algorithm decide on how to treat a packet. It is a kernel module, requiring elevated privileges to configure. The working operation of iptables is very simple. Each packet is stripped into various fields, and the rules from the table are applied to make a decision in terms of letting it go ahead, blocking it, or dropping it. For a given server role, iptables can be written only once, by taking into account all the packet acceptance and rejection scenarios, and would rarely be needed to change. While many production farms use iptables to introduce an additional layer of security, it is important to note that it puts an additional burden on the server's resources. Since every packet is stored temporarily and checked against a set of rules, it needs a considerable amount of computational power. Hence, iptables rules should not be very elaborate, but just adequate for the given network or application scenario. You can learn how to set up iptables on Ubuntu Linux, at <https://help.ubuntu.com/community/IptablesHowTo>

**ConnTrack:** This is another kernel-based module that falls under the net-filter framework. As an extension to iptables, ConnTrack essentially tracks the connection for all network sessions. It further tries to relate packets that formed a sensible and successful connection. ConnTrack operates at Layers 3 and 4, and creates useful information about each packet by reading its various fields. This can optionally be used further by iptables, to improve its effectiveness. For example, if the high-level protocol is HTTP, the packets are found to contain HTTP headers, as well as the session-based source and destination IP address, and service port information. If this data is made available by ConnTrack, it becomes easy for iptables to allow those packets without delving deep into them, thus saving precious (server) computational resources. The right approach is to have iptables and ConnTrack together.

**Source address verification:** One of the serious security attacks is packet spoofing, whereby attackers modify the source IP address to fool the destination host. As a result, it is rather difficult to detect and stop the spoofing attack.

Most Linux systems come with a built-in, but usually less known, feature called source address verification. It is a kernel feature that, when turned on, starts dropping packets that appear to be arriving from the internal network but in reality are not. Most of the latest kernels on distros such as Ubuntu and CentOS do support it; if your Linux distro does not, it is time to upgrade or migrate to a new distro. Modifying the *hosts.conf* file to add “*nospoof on*” is another level of defence to try. For smaller Linux networks, a nice utility called *arpwatch* is very useful for detection. *Arpwatch* keeps track of MAC and IP addresses, and records all changes—and can be scripted to alert administrators upon a possible attack. Scripting can also be done to go through network interface logs and look for anomalies with respect to source address forging.

**Anti-sniff:** Another serious type of attack is packet sniffing, wherein the network cards are put into promiscuous mode and packets are dumped for analysis to create an attack vector. All famous distros such as Ubuntu and CentOS do support anti-sniffing utilities, which monitor the network interface settings and ensure that promiscuous mode is not enabled. This effectively stops sniffers from working, thwarting further security attacks.

**SniffIt:** While the anti-sniffer is deployed in a FOSS network, it is important to see if it is functioning properly. For that, you need to simulate sniffing, and the SniffIT or DSNIFF utilities do that. Wireshark is another good example. The idea behind a sniffer is also to capture packet patterns that can eventually be fed into an intrusion detection system. Snort is a famous FOSS IDS system. DSNIFF is very effective in capturing SSL traffic.

## Beyond FOSS built-in security

As explained earlier, no single device or method can help you achieve 100 per cent security. Also, it is important to note that for some attacks such as packet sniffing, packet crafting, etc, there are no built-in security features available in an open source distro. All the methods explained here surely strengthen security, but they must be complemented with commercial-grade appliances and devices, to design a robust perimeter defence system. 

### By: Prashant Phatak

The author has over 20 years of experience in the field of IT hardware, networking, Web technologies and IT security. Prashant is MCSE and MCDBA certified, and is also an F5 load balancer expert. In the IT security world, he is an ethical hacker and net-forensic specialist. Prashant runs his own firm called Valency Networks in India (<http://www.valencynetworks.com>) providing consultancy in IT security design, security audits, infrastructure technology and business process management. He can be reached at [prashant@valencynetworks.com](mailto:prashant@valencynetworks.com).